CURENT Large-scale Testbed (LTB) Monthly Meeting

10/26/2026

CURENT, University of Tennessee, Knoxville

Leadership Team	Dr. Fran Li
Senior Advisor	Dr. Kevin Tomsovic, Dr. Hantao Cui, Dr. Xin Fang
Core Development	Dr. Hantao Cui, Dr. Xin Fang, Dr. Jinning Wang, Nicholas Parsly, Nick West, Zack Malkmus, Junjie Yin, Matthew Cannavaro, Hank Lin, Jian Han, Dr. Chenchen Li
Other Contributors	Dr. Qiwei Zhang, Dr. Buxin She, Dr. Hang Shuai

1. Summary

Community

- Homepage: https://ltb.curent.org/
- LinkedIn: https://www.linkedin.com/company/76122605
- YouTube: https://www.youtube.com/@curentltb
- GitHub: https://github.com/CURENT

Administration:

- 10/11-13 NAPS 2026 meeting
- 10/21 NASA visit CURENT

Development

- LLM with LTB AMS
  - Configuration file update
  - Revise parameter classification
  - How to ensure the LLM executes prompt correctly?
- LLM with LTB Andes

Configuration file update

A reviewed catalog can describe model edits; the compiler applies only declared templates.

FORMULATION CONTROL SPEC

control_id · operation_kind
scenario | formulation

routines · target_scope
DCOPF / ED / UC / ACOPF

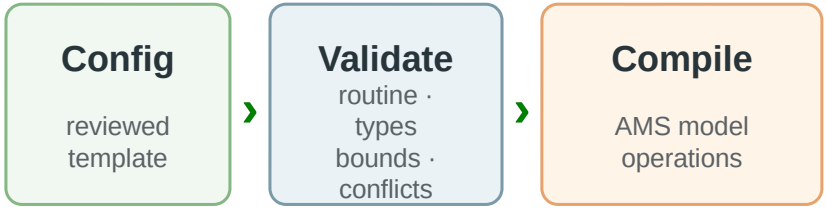
variables · objective_terms
what the template adds or changes

constraint_templates
named, versioned, reviewed blocks

parameters · bounds · units
typed values with validation rules

preconditions · conflicts · reset
safe composition and recovery

DECLARATION → MODEL CHANGE



Live AMS model
Only validated, routine-compatible edits are applied.

Design direction Replace one-off executor branches with configuration-driven, validated operations.

Revise parameter classification

Classify controls by their effect on data, constraints, and decision structure.

TYPE	CURRENT OPERATIONS	EFFECT ON THE OPTIMIZATION MODEL
Input values scenario	scale_pq_load — PQ_1 +10% set_line_rate — Line_1 rate_a = 0.9	Updates demand or line limits. Existing variables, objective terms, and constraint definitions remain; changed limits may still alter the feasible region.
Device / topology status scenario	set_line_status — open Line_1 set_generator_status — disable unit	Changes device availability, network state, and solve inputs. Results can shift without adding a decision-variable class.
Routine settings scenario	set_config_t — ED interval = 0.5 h	Changes the active routine's time setting and rebuilds its model; adds no custom objective term or constraint.
Existing constraint toggles formulation	set_constraint_status — disable plfub set_ramping_status — disable ED ramping	Changes which existing constraints participate in the solve. This is a formulation change; no new constraint is defined.
Add objective / constraint formulation	add_dispatch_cost set_total_generation_cap	First use adds an objective term or generation-cap constraint. Repeated use updates its coefficient or upper limit.
New variables + model rewrite formulation	replace_generator_with_ibr	DCOPF adds pibr / pcurt, availability limits, and cost terms, then rewrites nodal power balance (active-power scope).

Sources: formulation_controls.json:43–179 · parameter_controls.py:230–302 · formulation_edit.py:105–129

How to ensure the LLM executes a prompt correctly?

Treat execution as a verifiable transaction, not a reasoning claim.

1 REQUEST
“Set a = 0.9”
Target + value

›

2 PLAN
a := 0.9
Typed, scoped operation

›

3 PRECHECK
a exists · in range
Routine · units · preconditions

›

4 APPLY
Declared edit
One controlled model operation

›

5 READ BACK
Actual value = 0.9
Model diff + solve feasibility check

EXECUTION RECORD

Intent
target a · requested value 0.9

Applied state
read back from the live AMS object

Model effect
changed fields / objective / constraints

Outcome
solver status + feasibility checks

Block and ask when target, units, bounds, or post-apply state do not match.

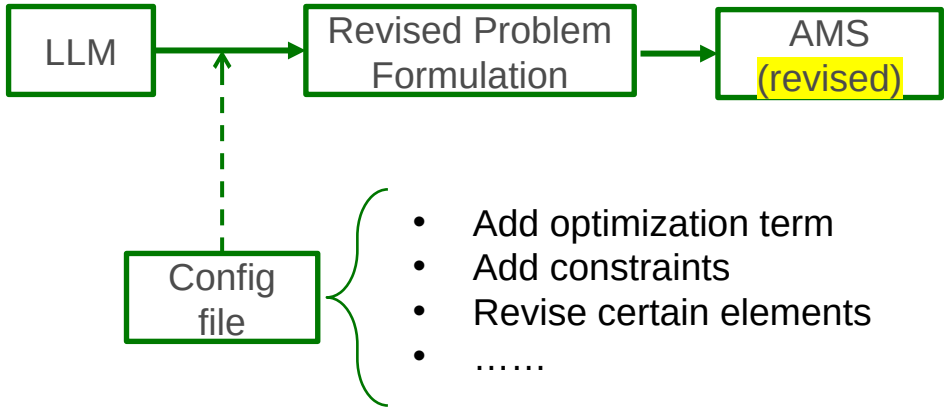
Revise the optimization problem formulation

Terminal (CLI):

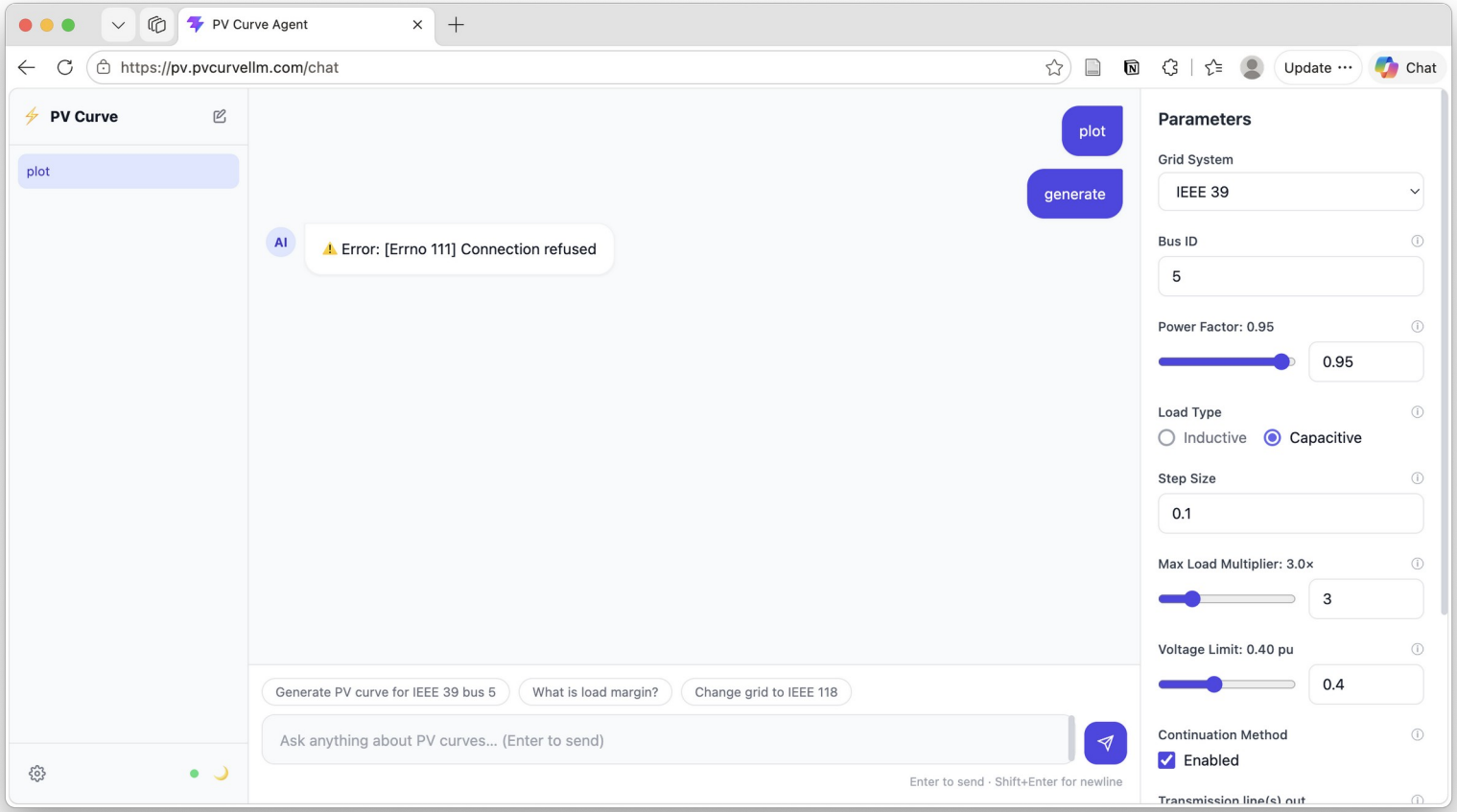
Question 1: doubling a generator's cost weight

--> Question 2 Replace PV_1 with an IBR with available power 1 pu, dispatch cost 1 dollar per pu, curtailment penalty 20 dollars per pu.

formulation_controls.json
- version
- constraint_labels
- ramping_constraints
- controls
  - id / label / description
  - enabled
  - routines
  - requires_constraints
  - parameters
  - prompt_examples
- unsupported_message



Hank’s PV curve interface

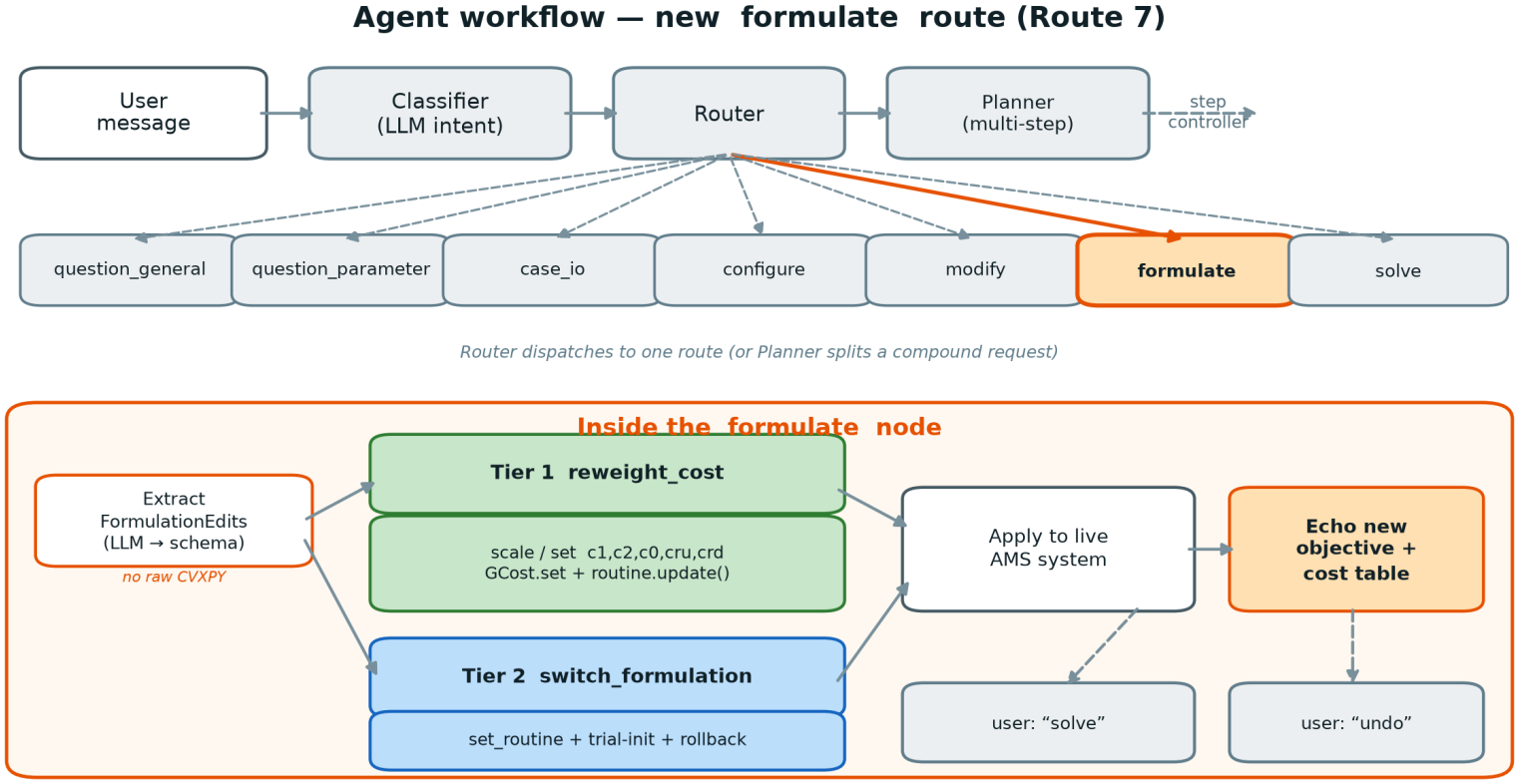


Revise the optimization problem formulation

Web UI: the formulation the agent edits

Revise the optimization problem formulation

Workflow: the new formulate route



How to verify or benchmark?

Benchmark: 9 / 9 checks pass on PJM 5-bus

benchmark · formulate route

```
$ python _validate_formulate.py
  Formulate-route validation & benchmark  ·  case: pjm 5-bus
```

	check	result
√ PASS	L0 DCP: convex passes, non-convex (c2<0) is flagged	is_dcp(convex)=True, is_dcp(c2<0)=False
√ PASS	L1 exact diff: only the target c1 changes, nothing else	PV_1 c1 1.2 → 2.4
√ PASS	L1 idempotence: scale x1 leaves the solution unchanged	|d obj|=0.00e+00
√ PASS	L1 undo round-trip: edit then undo restores the solution	|d obj|=0.00e+00
√ PASS	L3 scale-invariance: objective x3, argmin unchanged	obj 9.9325 → 29.7974 (3x=29.7974), max|dpg|=1.8e-15
√ PASS	L3 cost monotonicity: raising c1 weakly increases the optimum	obj 9.5360 → 10.0160
√ PASS	L3 relaxation monotonicity: dropping line limits weakly improves the optimum	obj 9.5360 → 5.1900
√ PASS	L3 analytical oracle: solved dispatch == merit order (before & after flip)	cheapest gen PV_2 forced to top of stack
√ PASS	L3 differential: cvxpy DCOPF vs PYPOWER DCOPF1 (min cost agrees)	cost_cvx=9.6560 cost_pp=9.6560 rel=4.2e-08

```
  9 passed, 0 failed  — exit code 0
```

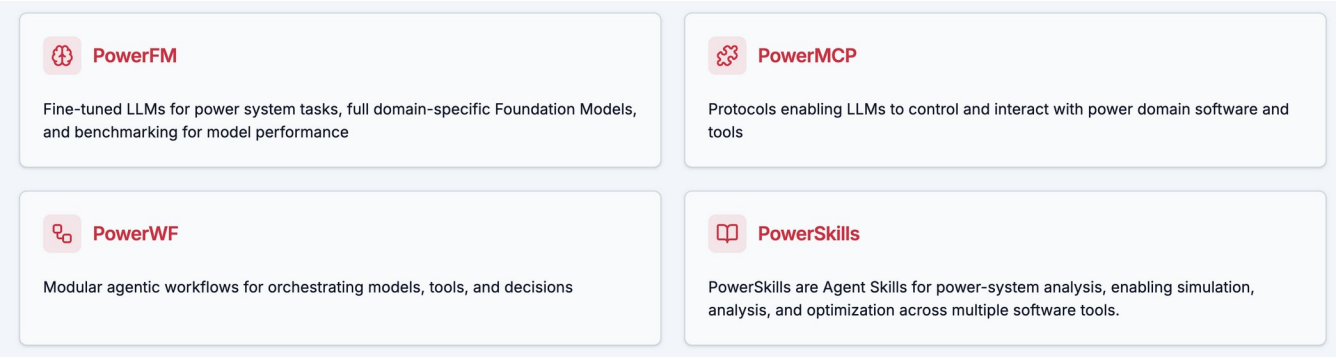
CURENT

How to verify or benchmark?

- **Layered verification:** Check user intent, formulation, implementation, and solution feasibility/optimality—not just solver success.
- **New benchmarks:** Combine expert-reviewed components, hand-solvable cases, and mathematical property tests (e.g., relaxing constraints cannot worsen the optimum).
- **Verifier evaluation:** Inject missing constraints, sign errors, and incorrect device mappings; supplement with cross-tool comparisons and expert review.

Comparison with peer works

PowerAgent – Harvard Dr. Le Xie



Module	Key Question	Main Function
PowerFM	Does the LLM **understand power systems**?	Domain foundation models, fine-tuned LLMs, and benchmarks
PowerMCP	Can the LLM **operate power-system software**?	Exposes PowerWorld, PSS/E, ANDES, pandapower, OpenDSS, etc. as LLM-callable tools
PowerWF	Does the LLM know **how to execute a task workflow**?	Orchestrates models, tools, decisions, and multi-step workflows
PowerSkills	Does the LLM know **how power engineers perform the task**?	Provides domain knowledge, engineering procedures, and task-specific instructions

CURENT

Comparison with peer works

Aspect	LLM_AMS	Harvard PowerAgent
Core Focus	AMS optimization models	Broad power-system software ecosystem
User Goal	Build or modify optimization problems, e.g., OPF and UC	Perform engineering tasks using power-system software
LLM Interaction Level	Deep **formulation/model level**	Primarily **software/tool level**
Tool Integration	Primarily AMS	PowerWorld, PSS/E, ANDES, pandapower, PyPSA, OpenDSS, etc.
Model Modification	A core target capability	Not a primary focus of PowerMCP
Objective / Constraints / Variables	Understand and modify mathematical formulations	Mainly use existing software-defined formulations
Verification	**Formulate → Verify → Repair**	Workflow checks, post-solution violation detection, and mitigation
Generality	Narrower software scope, **deeper model-level interaction**	Broader software scope, **shallower interaction with individual models**

PowerAgent emphasizes broad software-level integration, while LLM_AMS targets deeper interaction with optimization formulations.

CURENT

Acknowledgements

Contributors:











Users:



















Acknowledgements:



